The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Pranav Shukla	24BCE11525
Slot	**Date of Submission**
A13	30-3-2026

1. Introduction

Open source software is not only a technical model but also a social model of how knowledge grows. In proprietary systems, decisions, code access, and repair rights are concentrated with a company. In open source systems, these rights are distributed among users, maintainers, contributors, and organizations that cooperate around shared infrastructure.

For this audit, I selected Apache HTTP Server because it sits at the core of web history. It is one of the earliest examples of community-driven software becoming production-critical at global scale. It also reflects a practical open-source lesson: when an old project stagnates, a community can fork, patch, and build a healthier governance model.

This report examines Apache through four lenses. First, its origin and philosophical significance. Second, its Linux footprint and operational behavior. Third, the broader FOSS ecosystem around it. Fourth, a realistic comparison against a proprietary alternative (Microsoft IIS). Finally, the report

documents five shell scripts that demonstrate Linux and Bash fundamentals connected to the open-source theme.

2. Part A - Origin and Philosophy (Units 1 and 2)

A1. The problem Apache was created to solve

In the mid-1990s, the web expanded quickly, but web server tooling did not evolve at the same pace. NCSA HTTPd was one of the main servers used at that time, but active development slowed. Administrators still needed fixes for security, stability, and performance, so many teams began maintaining local patches independently.

A group of webmasters and developers started sharing those patches in a coordinated way. Instead of waiting for centralized updates, they created a collaborative workflow to merge improvements. This became the Apache Group. The early joke that Apache was "a patchy server" captured the practical origin: it was born from real operations pain, not from a marketing plan.

The core problem Apache solved was reliability and control for web administrators. Before Apache, teams had fewer options to adapt server behavior to their own needs. Apache introduced modularity, transparent source access, and collective maintenance. If a feature was missing, an organization could build a module. If a bug appeared, anyone with skill could inspect code and propose a fix.

Equally important is why the creators shared it openly. At that stage of the web, interoperability mattered more than lock-in. The internet could not grow on top of many closed, incompatible islands. Apache offered a practical path where institutions, startups, and individuals could run a standards-oriented web server without licensing barriers.

From a software history perspective, Apache represents a transition from local patch culture to global open governance. It proved that distributed volunteer and enterprise contributors could produce infrastructure software trusted by millions of systems.

A2. The license - what it actually says

Apache HTTP Server uses the Apache License 2.0. This is a permissive free and open-source license with explicit patent grants and clear redistribution conditions.

The four freedoms and Apache License 2.0

The classic four freedoms of free software are:

1. Freedom to run the program for any purpose.
2. Freedom to study how the program works and change it.
3. Freedom to redistribute copies.
4. Freedom to distribute modified versions.

Apache 2.0 grants these freedoms in practice. Users can run, inspect, modify, and redistribute code, including modified versions. The license requires preservation of notices and disclaimers, and it includes patent terms that protect downstream users from certain patent aggression.

Can a company modify Apache and sell it without sharing changes?

Yes, a company can modify Apache and sell a product containing it without publishing its modified source code. Apache 2.0 is permissive, not copyleft. The company must satisfy license obligations (such as preserving notices and license text), but it is not forced to release private modifications.

This is a major difference compared to GPL-style copyleft licenses. Apache encourages broad adoption, including in commercial contexts, by reducing reciprocity obligations.

GPL vs MIT, and which I would choose

GPL and MIT both allow source sharing, but they drive different contribution economics.

- GPL: Strong copyleft. If distributed binaries include GPL code or derivatives, source obligations are triggered. This protects openness of downstream modifications.
- MIT: Very permissive. Minimal requirements (mainly copyright and license notice retention). Modified code can be relicensed in closed products.

If I were building a tool where long-term community reciprocity is critical (for example, a security library or educational platform), I would choose GPL or another copyleft license to keep improvements flowing back.

If I were building a component where ecosystem adoption is the top priority (for example, integration SDKs), I would choose MIT or Apache 2.0.

For a web server-scale platform used in mixed enterprise contexts, Apache 2.0 is a balanced choice because it supports both open collaboration and commercial adoption.

"Free beer" vs "freedom"

"Free as in free beer" means zero cost. "Free as in freedom" means user rights.

Apache is often free in both senses, but the deeper value is freedom. Even if a managed Apache-based service is paid, users still benefit from an open implementation standard and a codebase that can be audited, forked, or self-hosted. The ability to move away from a vendor without losing technical control is a freedom argument, not a price argument.

A3. The ethics of open source

Should all software be open source?

Argument for: critical digital infrastructure should be inspectable and accountable. Open source supports reproducibility, peer review, security transparency, and educational access. Public sector software and high-impact civic systems especially benefit from openness because citizens bear the risk of hidden failures.

Argument against: some software contexts involve sensitive threat models, regulated datasets, or business constraints where full code openness may introduce practical and legal complications. Also, open source does not automatically guarantee quality, governance, or sustainability.

My position: not all software must be open source, but all software that governs public trust should move toward greater transparency and auditability.

Is it ethical to profit from community labor?

It can be ethical, but only under fair exchange. Companies such as Red Hat built sustainable businesses by adding enterprise support, integration, and long-term maintenance while contributing heavily upstream. That model returns value to the commons.

It becomes ethically questionable when organizations extract value without meaningful contribution, shift maintenance burden to volunteers, or use open-source branding while avoiding community governance norms.

A useful ethical test is reciprocity: does the company only take code, or does it also return patches, funding, documentation, bug triage, mentorship, and responsible disclosure practices?

Standing on the shoulders of giants

In software, this phrase means every project inherits from prior tools, protocols, and ideas. Open source formalizes this inheritance by making code visible and reusable.

Some critics say reuse reduces originality. I disagree. Reuse is not the opposite of innovation; it is its acceleration layer. Teams spend less time rebuilding basic components and more time solving new problems. Apache itself is evidence: by standardizing reliable web serving, it enabled others to innovate on frameworks, applications, and cloud architectures.

---

3. Part B - Linux Footprint (Unit 2)

This section documents how Apache HTTP Server appears and behaves on Linux systems.

Test environment used

- Distribution: Fedora Linux 43
- Kernel: [Fill output of uname -r]
- Installation method: dnf (RPM package management)
- Date tested: [Fill test date]
- Environment type: [Physical machine or VM]

Installation method

For this project, Fedora Linux 43 was used. Apache package name on Fedora is httpd.

Commands used on Fedora:

sudo dnf update -y sudo dnf install -y httpd

From an open-source distribution perspective, package repositories are themselves community pipelines. Security fixes travel from upstream maintainers to distro maintainers to end users through signed package updates.

Key directories and files

Typical Apache footprint on Linux:

Component	Debian/Ubuntu Path	RHEL/Fedora Path	Purpose
Main binary	/usr/sbin/apache2	/usr/sbin/httpd	Web server executable
Main config	/etc/apache2/apache2.conf	/etc/httpd/conf/httpd.conf	Core server configuration
Site configs	/etc/apache2/sites-available	/etc/httpd/conf.d	Virtual host definitions
Modules	/usr/lib/apache2/modules	/etc/httpd/modules or /usr/lib64/httpd/modules	Loadable feature modules
Logs	/var/log/apache2	/var/log/httpd	Access and error logs
Web root	/var/www/html	/var/www/html	Default document root

This layout reflects Linux design conventions: binaries in /usr, configuration in /etc, variable runtime logs in /var/log, and web content under /var/www.

User and group model

Apache usually runs worker processes as:

- apache (Fedora 43)

Why this matters: least privilege. The service should not run as root for request handling. Root may be used briefly for startup and privileged binding, but active request processing should run with restricted permissions. This containment reduces the blast radius of potential vulnerabilities.

Verification commands:

ps aux | grep httpd id apache

Service management

Apache is managed through systemd on modern Linux.

Fedora commands used:

sudo systemctl start httpd sudo systemctl stop httpd sudo systemctl restart httpd sudo systemctl status httpd sudo systemctl enable httpd

Operationally, this gives a standard control plane for boot behavior, logs (via journald and files), and dependency orchestration.

Update and patch model

Apache vulnerabilities are tracked through CVEs and upstream security advisories. Patch flow is typically:

1. Issue discovery and responsible disclosure.
2. Upstream patch development and review.
3. Release publication by Apache maintainers.
4. Distro maintainers backport or package updates.
5. User systems receive updates through dnf on Fedora.

Commands:

Fedora:

sudo dnf check-update sudo dnf upgrade

This process demonstrates a practical open-source security strength: patch visibility and distributed review.

Evidence placeholders (add your own screenshots)

- Screenshot B1: Apache package installation command and success output
- Screenshot B2: systemctl status output
- Screenshot B3: ls -l or tree output of config and log directories
- Screenshot B4: ps aux showing service user account

---

4. Part C - The FOSS Ecosystem (Units 3 and 4)

Apache does not exist in isolation. Its utility comes from its integration with standards, libraries, and companion tools.

Core dependencies and supporting tools

Common Apache build/runtime dependencies include:

- APR and APR-util (Apache Portable Runtime)
- PCRE or PCRE2 for pattern matching
- OpenSSL for TLS support
- zlib for compression-related features
- System libraries for threading, sockets, and file I/O

In real deployments, Apache is often connected with:

- PHP modules or PHP-FPM integration
- Python via WSGI interfaces
- Reverse proxy features to app servers
- Databases (MySQL/MariaDB, PostgreSQL) in web application stacks

What Apache enabled or inspired

Apache enabled early dynamic web hosting at scale. Shared hosting providers, educational institutions, and SMEs could deploy websites with low cost and strong flexibility.

It also normalized module-based server customization. This architecture influenced how administrators think about composable web infrastructure: load balancing, URL rewriting, TLS termination, authentication, and caching could be managed as modular capabilities.

Apache did not remain the only dominant web server; it also catalyzed competition. Nginx and others emerged with alternative performance models, but this competition itself is part of healthy open ecosystems.

Apache and the LAMP stack

In LAMP, Apache is the "A":

- L = Linux
- A = Apache
- M = MySQL/MariaDB
- P = PHP/Perl/Python

Historically, LAMP lowered entry barriers for web startups and student developers. A complete internet-ready stack could be assembled from open components with broad documentation and community support.

Even in cloud-native contexts where architectures evolve, Apache remains relevant in reverse proxy, legacy support, intranet hosting, and compliance-controlled environments.

Community and governance

Apache HTTP Server is governed under the Apache Software Foundation (ASF), with project management committees, mailing-list culture, issue tracking workflows, and merit-based contribution norms.

Community spaces include:

- Apache mailing lists
- ASF issue trackers and repository workflows
- ApacheCon events
- Community forums and distro communities

Governance is an important open-source feature. Good projects are not only code repositories; they are social systems with review standards, release policies, and conflict resolution mechanisms.

---

5. Part D - Open Source vs Proprietary (Critical Analysis)

Chosen comparison: Apache HTTP Server vs Microsoft IIS.

Dimension	Apache HTTP Server (Open Source)	Microsoft IIS (Proprietary Alternative)
Cost	No license fee; operational cost depends on infra and team skills	Typically tied to Windows Server licensing and enterprise contracts
Security (auditability)	Source code can be audited by anyone; broad public scrutiny	Source not publicly auditable; users rely on vendor disclosures
Support and reliability	Community plus paid support ecosystem; mature and stable	Strong centralized vendor support and enterprise tooling
Freedom to modify	High freedom to modify behavior and modules	Customization is constrained by vendor platform boundaries
Community vs corporate control	Multi-stakeholder open governance	Primarily corporate roadmap control
Integration footprint	Excellent in Linux/Unix ecosystems and mixed environments	Excellent in Windows-centric enterprise environments
Learning and transparency	Strong for education and debugging due to open internals	Easier guided setup in some enterprise stacks, but internals are opaque
Overall verdict	Best when openness, portability, and controllability are priorities	Best when organization is deeply standardized on Microsoft stack

Two-paragraph verdict

For real-world deployments, I would recommend Apache when an organization values transparency, platform flexibility, and long-term control over infrastructure behavior. Its maturity, broad documentation, and ecosystem integrations make it suitable for both small and large deployments. It is especially strong in Linux-native environments and in teams that want to tune server behavior beyond default presets.

I would contribute to Apache indirectly through documentation improvements, issue triage, and small patches as my first steps. Contributing to mature infrastructure projects is less about single heroic commits and more about sustained participation in review and maintenance culture. This aligns with the core open-source ethic: software quality is a collective responsibility.

---

6. Shell Script Tasks Documentation

This section documents each script, key shell concepts used, and where to place screenshots.

Script 1 - System Identity Report

File: scripts/01_system_identity_report.sh

Purpose:

- Displays Linux distribution and kernel details.
- Shows current user and home directory.
- Prints uptime and current date/time.
- Includes a Linux license note.

Concepts used:

- Variables
- Command substitution $( )
- Conditional file check
- Formatted output using echo

How to run:

./scripts/01_system_identity_report.sh

Screenshot placeholder:

- Screenshot S1: Terminal output of Script 1

Explanation:

This script acts like a welcome report for the audit environment. It collects system information from standard Linux commands and prints them in a readable block. It also demonstrates why open-source transparency matters: even basic OS identity and licensing can be inspected without proprietary tooling.

Script 2 - FOSS Package Inspector

File: scripts/02_foss_package_inspector.sh

Purpose:

- Checks whether a selected package is installed.
- Pulls metadata such as version and license/description.
- Uses a case statement for one-line package philosophy notes.

Concepts used:

- if-then-else
- case statement
- rpm and dpkg compatibility logic
- grep filtering with pipes

How to run:

./scripts/02_foss_package_inspector.sh httpd ./scripts/02_foss_package_inspector.sh mariadb
./scripts/02_foss_package_inspector.sh git

Screenshot placeholder:

- Screenshot S2: Script 2 checking installed package metadata

Explanation:

This script connects package management with open-source literacy. It does not just ask "is software installed" but also surfaces license and purpose context, reinforcing that package managers are practical gateways to open software distribution.

Script 3 - Disk and Permission Auditor

File: scripts/03_disk_permission_auditor.sh

Purpose:

- Audits key system directories.
- Reports owner/group/permissions and disk usage.
- Verifies Apache configuration directory and permissions.

Concepts used:

- Array and for loop
- ls -ld parsing with awk

- du and cut for size extraction
- if checks for directory existence

How to run:

./scripts/03_disk_permission_auditor.sh

Screenshot placeholder:

- Screenshot S3: Script 3 output with directory audit lines

Explanation:

This script demonstrates operational Linux awareness. In production systems, permission mistakes can become security incidents. By combining size and permission checks, the script shows how routine auditing can be automated with simple shell tools.

Script 4 - Log File Analyzer

File: scripts/04_log_file_analyzer.sh

Purpose:

- Reads a log file line by line.
- Counts case-insensitive keyword matches.
- Retries when file is empty.
- Prints the last five matching lines.

Concepts used:

- Command-line arguments ($1 and optional $2)
- while IFS= read loop
- if-then matching logic
- Counters and arithmetic expansion

How to run:

./scripts/04_log_file_analyzer.sh /var/log/httpd/access_log GET ./scripts/04_log_file_analyzer.sh /var/log/httpd/error_log error

Screenshot placeholder:

- Screenshot S4: Script 4 keyword count and last matching lines

Explanation:

Logs are one of the most direct forms of system truth. This script demonstrates how Bash can provide lightweight observability without external platforms. The retry logic also models defensive scripting by handling empty-input edge cases.

Script 5 - Open Source Manifesto Generator

File: scripts/05_open_source_manifesto_generator.sh

Purpose:

- Interactively collects three user inputs.
- Generates a personalized open-source statement.
- Writes output to a timestamped text file.

Concepts used:

- read for interactive input
- String interpolation
- File output using > and >>
- date command
- Alias concept in comments

How to run:

./scripts/05_open_source_manifesto_generator.sh

Screenshot placeholder:

- Screenshot S5: Script 5 interaction and generated file preview

Explanation:

This creative script links technical scripting to personal philosophy. It shows how shell tools can automate not only system tasks but also text generation workflows. The output file can be archived as part of reflective coursework evidence.

---

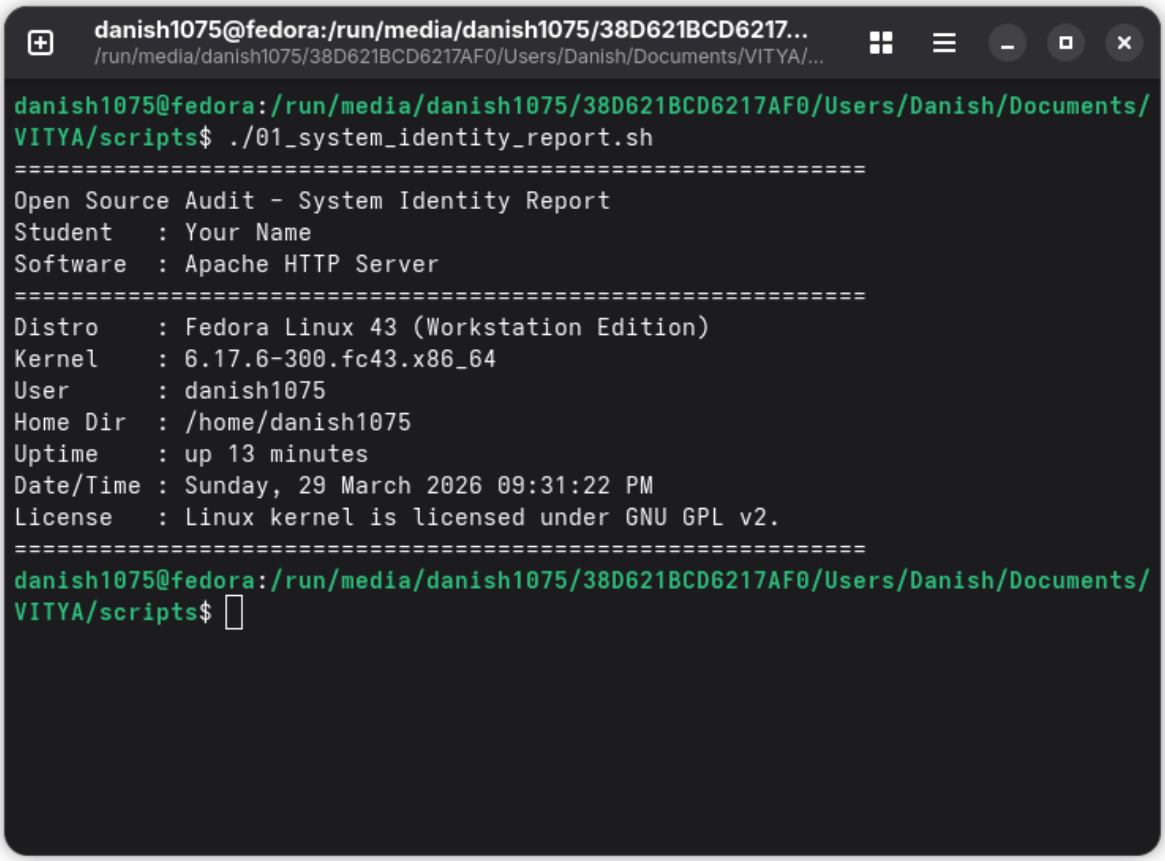
7. Script Source Code Appendix

Important: Keep this section synchronized with actual files in scripts/. If you edit any script, update this appendix before final submission.

7.1 Script 1 Code

```bash
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software (OSS NGMC)
# Purpose: Show core Linux identity details for the audit environment.

# ------------------------------
# Student and project variables
# ------------------------------
STUDENT_NAME="Your Name"
SOFTWARE_CHOICE="Apache HTTP Server"

# ------------------------------
# System information collection
# ------------------------------
KERNEL="$(uname -r)"
USER_NAME="$(whoami)"
HOME_DIR="$HOME"
UPTIME="$(uptime -p)"
NOW="$(date '+%A, %d %B %Y %I:%M:%S %p')"

# Read distro name from os-release if available.
if [ -r /etc/os-release ]; then
    DISTRO_NAME="$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2- | tr -d
'"')"
else
    DISTRO_NAME="$(uname -s)"
fi

# Linux kernel license note for this report context.
OS_LICENSE_NOTE="Linux kernel is licensed under GNU GPL v2."

# ------------------------------
# Display
# ------------------------------
echo "============================================================"
echo "Open Source Audit - System Identity Report"
echo "Student   : $STUDENT_NAME"
echo "Software  : $SOFTWARE_CHOICE"
```

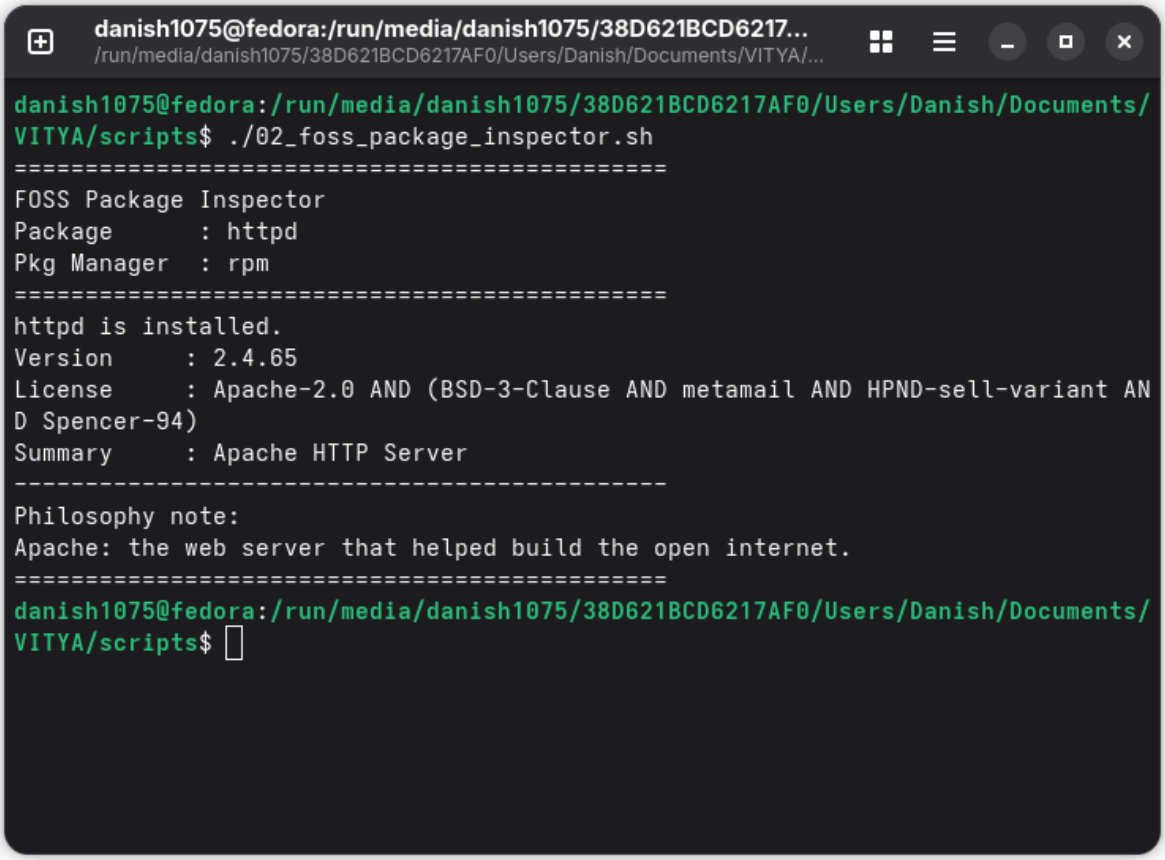
```
echo "============================================================"
echo "Distro    : $DISTRO_NAME"
echo "Kernel    : $KERNEL"
echo "User      : $USER_NAME"
echo "Home Dir  : $HOME_DIR"
echo "Uptime    : $UPTIME"
echo "Date/Time : $NOW"
echo "License   : $OS_LICENSE_NOTE"
echo "============================================================"
```



7.2 Script 2 Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Course: Open Source Software (OSS NGMC)
# Purpose: Check whether a package is installed and print basic metadata.

# Default package is Apache HTTP Server package name for RPM systems.
# You can pass a package name as argument, for example:
# ./02_foss_package_inspector.sh httpd
PACKAGE="${1:-httpd}"

# Detect package manager to choose correct commands.
if command -v rpm >/dev/null 2>&1; then
    PKG_MANAGER="rpm"
elif command -v dpkg >/dev/null 2>&1; then
    PKG_MANAGER="dpkg"
else
    echo "Error: neither rpm nor dpkg was found on this system."
    exit 1
fi

echo "============================================="
echo "FOSS Package Inspector"
echo "Package      : $PACKAGE"
echo "Pkg Manager  : $PKG_MANAGER"
echo "============================================="

# Check if package is installed and print metadata.
if [ "$PKG_MANAGER" = "rpm" ]; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "$PACKAGE is installed."
        rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary'
    else
        echo "$PACKAGE is NOT installed."
    fi
else
    if dpkg -l | grep -E "^ii[[:space:]]+$PACKAGE[[:space:]]" >/dev/null; then
        echo "$PACKAGE is installed."
```

```
        dpkg -s "$PACKAGE" | grep -E '^(Version|Maintainer|Description):'
    else
        echo "$PACKAGE is NOT installed."
    fi
fi

echo "----------------------------------------------------------------------"
echo "Philosophy note:"

# Case statement to describe packages briefly.
case "$PACKAGE" in
    httpd|apache2)
        echo "Apache: the web server that helped build the open internet."
        ;;
    mysql|mariadb|mariadb-server)
        echo "MySQL/MariaDB: open databases that power applications at global
scale."
        ;;
    firefox|firefox-esr)
        echo "Firefox: a community-backed browser defending an open web."
        ;;
    vlc)
        echo "VLC: open multimedia software proving interoperability matters."
        ;;
    git)
        echo "Git: distributed version control built for collaborative
development."
        ;;
    python3|python)
        echo "Python: a language advanced by community process and shared
tooling."
        ;;
    *)
        echo "This package is part of the wider FOSS ecosystem of shareable
tools."
        ;;
esac
echo "=============================================="
```

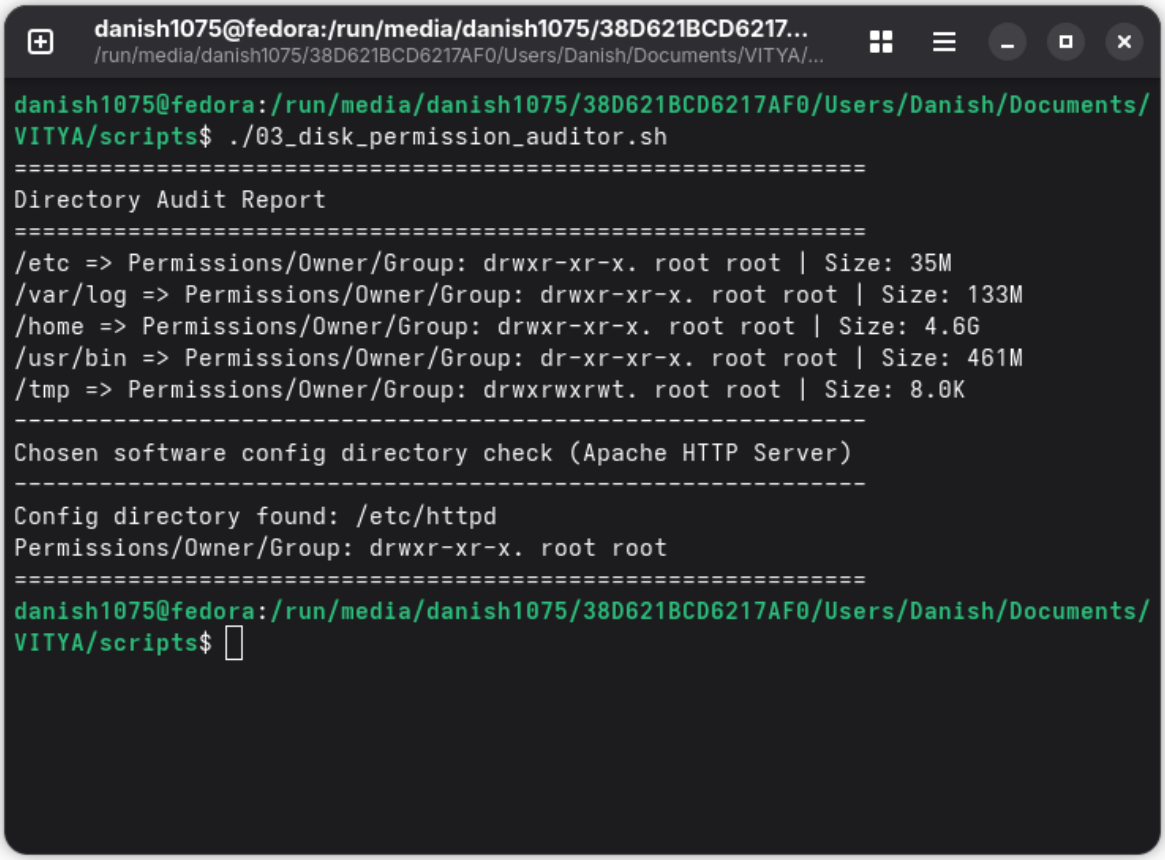

danish1075@fedora:/run/media/danish1075/38D621BCD6217...
/run/media/danish1075/38D621BCD6217AF0/Users/Danish/Documents/VITYA/...
danish1075@fedora:/run/media/danish1075/38D621BCD6217AF0/Users/Danish/Documents/VITYA/scripts$ ./02_foss_package_inspector.sh
==========
FOSS Package Inspector
Package : httpd
Pkg Manager : rpm
==========
httpd is installed.
Version : 2.4.65
License : Apache-2.0 AND (BSD-3-Clause AND metamail AND HPND-sell-variant AND Spencer-94)
Summary : Apache HTTP Server

Philosophy note:
Apache: the web server that helped build the open internet.
==========
danish1075@fedora:/run/media/danish1075/38D621BCD6217AF0/Users/Danish/Documents/VITYA/scripts$

7.3 Script 3 Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: [Your Name] | Course: Open Source Software (OSS NGMC)
# Purpose: Audit key Linux directories for permissions, owner/group, and size.

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "============================================================"
echo "Directory Audit Report"
echo "============================================================"

# Loop through important directories and collect permission/size details.
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS="$(ls -ld "$DIR" | awk '{print $1, $3, $4}')"
        SIZE="$(du -sh "$DIR" 2>/dev/null | cut -f1)"
        echo "$DIR => Permissions/Owner/Group: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system."
    fi
done

echo "------------------------------------------------------------------------------------------------"
echo "Chosen software config directory check (Apache HTTP Server)"
echo "------------------------------------------------------------------------------------------------"

# Apache config path differs by distro family.
if [ -d "/etc/httpd" ]; then
    CONFIG_DIR="/etc/httpd"
elif [ -d "/etc/apache2" ]; then
    CONFIG_DIR="/etc/apache2"
else
    CONFIG_DIR=""
fi

if [ -n "$CONFIG_DIR" ]; then
    CFG_PERMS="$(ls -ld "$CONFIG_DIR" | awk '{print $1, $3, $4}')"
```

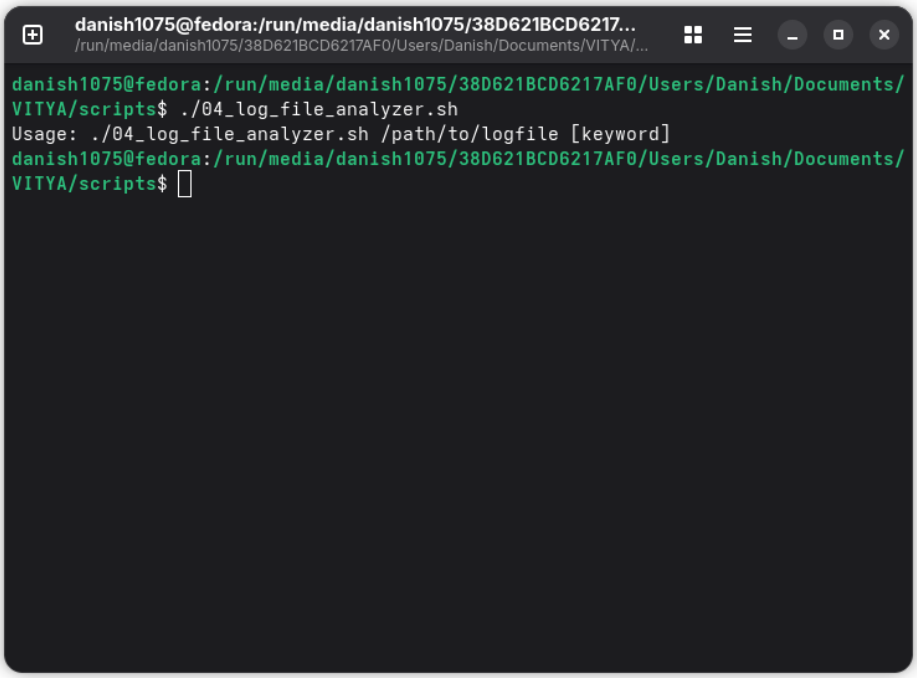
```
    echo "Config directory found: $CONFIG_DIR"
    echo "Permissions/Owner/Group: $CFG_PERMS"
else
    echo "Apache config directory not found (/etc/httpd or /etc/apache2)."
fi

echo "============================================================"
```



7.4 Script 4 Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: [Your Name] | Course: Open Source Software (OSS NGMC)
# Usage: ./04_log_file_analyzer.sh /path/to/logfile [keyword]
# Purpose: Count keyword matches in a log file and print recent matching lines.

LOGFILE="$1"
KEYWORD="${2:-error}"
COUNT=0
RETRIES=0
MAX_RETRIES=3

if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 /path/to/logfile [keyword]"
    exit 1
fi

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style retry for empty logs: check, wait, retry up to MAX_RETRIES.
while [ ! -s "$LOGFILE" ]; do
    RETRIES=$((RETRIES + 1))
    echo "Warning: $LOGFILE is empty. Retry $RETRIES/$MAX_RETRIES in 2
seconds..."

    if [ "$RETRIES" -ge "$MAX_RETRIES" ]; then
        echo "No data available after retries. Exiting."
        exit 1
    fi

    sleep 2
done

# Read file line by line and count matches case-insensitively.
```

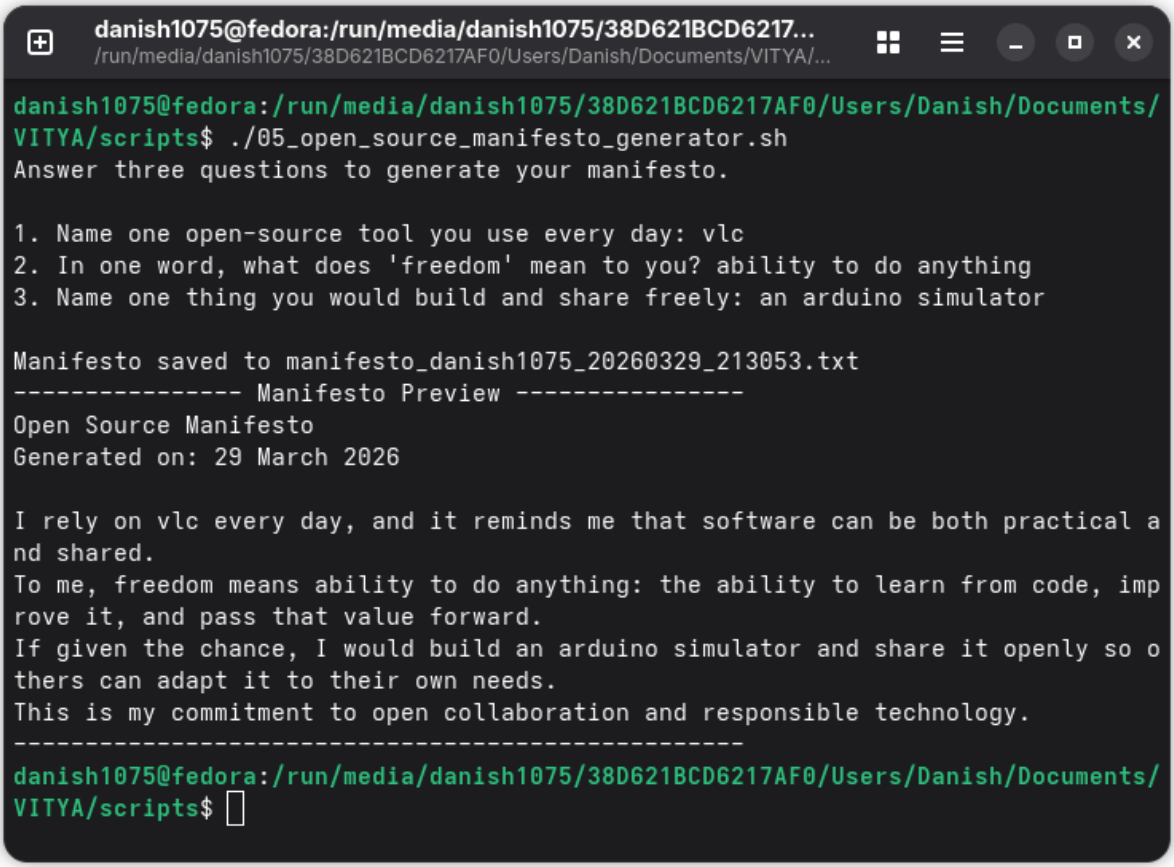
```
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "============================================================"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "------------------------------------------------------------"
echo "Last 5 matching lines:"

# Show only recent matching lines from the tail of the log for readability.
MATCHES="$(tail -n 2000 "$LOGFILE" | grep -i "$KEYWORD" | tail -n 5)"

if [ -n "$MATCHES" ]; then
    echo "$MATCHES"
else
    echo "No matching lines found."
fi

echo "============================================================"
```



7.5 Script 5 Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Course: Open Source Software (OSS NGMC)
# Purpose: Collect user ideas and generate a personalized manifesto text file.

echo "Answer three questions to generate your manifesto."
echo

# Interactive prompts.
read -r -p "1. Name one open-source tool you use every day: " TOOL
read -r -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -r -p "3. Name one thing you would build and share freely: " BUILD

DATE="$(date '+%d %B %Y')"
OUTPUT="manifesto_$(whoami)_$(date '+%Y%m%d_%H%M%S').txt"

# Alias concept note:
# Example alias that many Linux users define in ~/.bashrc:
# alias ll='ls -alF'

# Write manifesto header (overwrite file if it exists with same name).
echo "Open Source Manifesto" > "$OUTPUT"
echo "Generated on: $DATE" >> "$OUTPUT"
echo >> "$OUTPUT"

# Compose a short personalized paragraph and append to file.
echo "I rely on $TOOL every day, and it reminds me that software can be both
practical and shared." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM: the ability to learn from code, improve it,
and pass that value forward." >> "$OUTPUT"
echo "If given the chance, I would build $BUILD and share it openly so others can
adapt it to their own needs." >> "$OUTPUT"
echo "This is my commitment to open collaboration and responsible technology." >>
"$OUTPUT"

echo
echo "Manifesto saved to $OUTPUT"
```

```
echo "------------------------ Manifesto Preview ------------------------"
cat "$OUTPUT"
echo "-------------------------------------------------------------------------------"
```



---

8. Conclusion

This audit shows that Apache HTTP Server is not just a software package but a historical and philosophical case study in open-source collaboration. It solved a concrete technical problem at the right time, evolved under community governance, and became foundational infrastructure for the public web.

From a Linux operations perspective, Apache follows predictable and auditable conventions for service management, configuration, and patching. From a social perspective, its permissive licensing and broad ecosystem integration illustrate how open infrastructure can support both community goals and commercial deployment models.

The shell scripts in this submission reinforce that open source is learned not only by reading theory but by practicing at the command line. Through scripting, package inspection, permissions auditing, log analysis, and reflective text generation, this project links philosophy to practical systems work.

---

9. References

1. GNU Project - The Free Software Definition
2. Open Source Initiative - The Open Source Definition
3. Apache Software Foundation - Apache HTTP Server Project Documentation
4. SPDX License List
5. Linux man pages and distribution package documentation
6. The Linux Command Line by William Shotts

---

11. Fedora 43 Run Order (For Reproducible Demo)

Use this exact sequence during testing and screenshot capture:

1. cat /etc/fedora-release
2. sudo dnf update -y
3. sudo dnf install -y httpd curl
4. sudo systemctl enable --now httpd
5. sudo systemctl status httpd
6. chmod +x scripts/*.sh
7. ./scripts/01_system_identity_report.sh
8. ./scripts/02_foss_package_inspector.sh httpd
9. ./scripts/03_disk_permission_auditor.sh
10. curl -I http://localhost
11. curl -I http://localhost/nonexistent
12. ./scripts/04_log_file_analyzer.sh /var/log/httpd/access_log GET
13. sudo ./scripts/04_log_file_analyzer.sh /var/log/httpd/error_log error
14. ./scripts/05_open_source_manifesto_generator.sh